

Procesos en Linux

Creación, señales y finalización de procesos

Basado en “Advanced Linux Programming” — Mitchell, Oldham & Samuel (New Riders, 2001)



```
$ whoami
system-programming

$ ps -e -o pid,ppid,comm
PID  PPID  COMMAND
   1     0    init
  340    1   bash
  521   340   ps
```

Agenda

1

Introducción a los procesos

PID, PPID, el árbol de procesos, ps y kill

§3.1

2

Creación de procesos

La función `system()`, y el patrón `fork() + exec()`

§3.2

3

Señales

Comunicación asíncrona: tipos de señales y `sigaction()`

§3.3

4

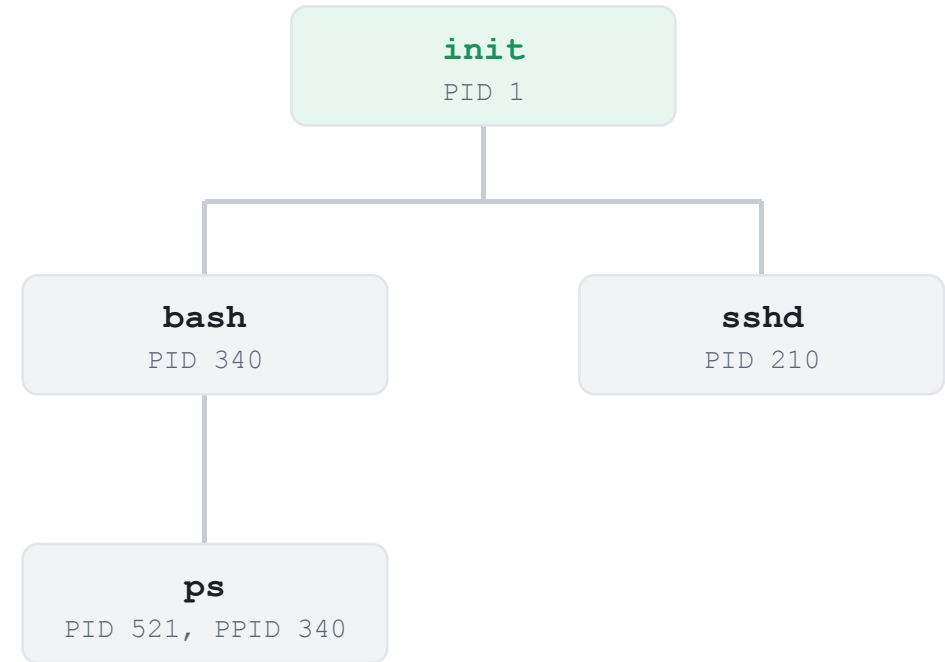
Finalización y terminación

`exit()`, `wait()`, procesos zombie y `SIGCHLD`

§3.4

¿Qué es un proceso?

- Una instancia en ejecución de un programa.
- Dos terminales abiertas ejecutan el mismo programa, pero son dos procesos distintos.
- Todo proceso tiene un proceso padre — excepto uno.
- `init` (PID 1) es la raíz del árbol de procesos.
- API en `<unistd.h>`: `getpid()` y `getppid()`.



El árbol de procesos

Identificar y controlar procesos

```
#include <unistd.h>
#include <stdio.h>

int main(void) {
    printf("PID:  %d\n", (int)getpid());
    printf("PPID: %d\n", (int)getppid());
    return 0;
}
```

```
ps -e -o pid,ppid,cmd
```

Lista todos los procesos activos del sistema.

```
kill <pid>
```

Envía SIGTERM — solicita que el proceso termine.

```
kill -KILL <pid>
```

Envía SIGKILL — fuerza la terminación inmediata.

Las señales SIGTERM y SIGKILL se explican en la Sección 3.

La forma simple: `system()`

- `system("ls -l /")` ejecuta un comando desde el programa.
- Internamente crea un subprocesso que ejecuta `/bin/sh`.
- Retorna el código de salida del comando ejecutado.

```
int main(void) {  
    return system("ls -l /");  
}
```

✓ Ventaja

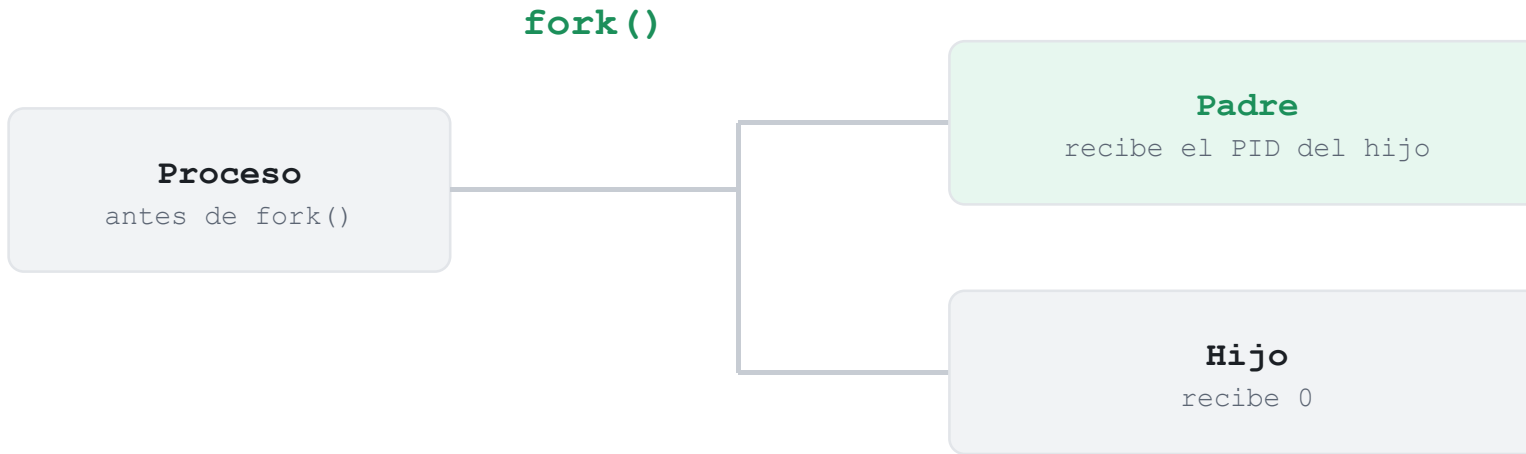
Simple de usar: un solo llamado.

✗ Desventajas

Ineficiente, depende del shell del sistema y tiene riesgos de seguridad.

→ Por eso se prefiere `fork()` + `exec()`

fork() : duplicando un proceso



```
pid_t child_pid = fork();

if (child_pid != 0)
    // proceso padre (child_pid = PID del hijo)
else
    // proceso hijo (child_pid == 0)
```

La familia exec

exec*() reemplaza el programa del proceso actual. Si tiene éxito, nunca retorna.



Busca el programa en PATH
`execvp · execlp`



Argumentos como arreglo (vector) terminado en NULL
`execv · execvp · execve`



Argumentos como lista fija (varargs)
`execl · execlp · execl`

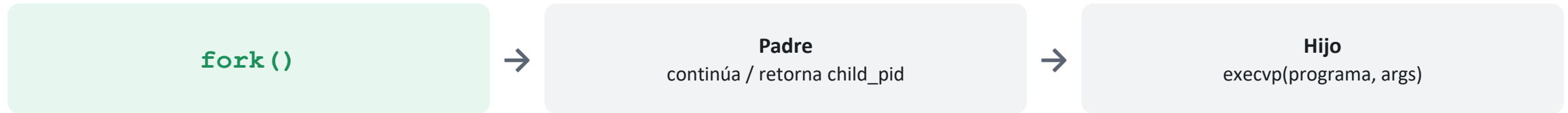


Permite especificar variables de entorno
`execve · execl`



```
execvp("ls", arg_list);
```

fork + exec: el patrón spawn



```
pid_t spawn(char* program, char** arg_list) {  
    pid_t child_pid = fork();  
  
    if (child_pid != 0)  
        return child_pid;           // proceso padre  
  
    execvp(program, arg_list);       // proceso hijo  
    abort();                         // solo si execvp falla  
}
```

Bonus: nice y renice ajustan la prioridad (niceness) de un proceso.

Señales: interrupciones asíncronas

Una señal es un mensaje especial enviado a un proceso. Se procesa de inmediato, sin esperar a que termine la instrucción actual: por eso son asíncronas.

SIGTERM

Pide terminar (señal por defecto de kill)

SIGKILL

Termina de forma forzada e inevitable

SIGSEGV

Acceso inválido a memoria

SIGUSR1 / SIGUSR2

Definidas para uso de la aplicación

SIGHUP

Recargar configuración

SIGCHLD

Un proceso hijo terminó

Manejo de señales con `sigaction()`

```
sig_atomic_t contador = 0;

void handler(int signum) {
    ++contador;
}

sa.sa_handler = &handler;
sigaction(SIGUSR1, &sa, NULL);
```

Disposición de la señal

SIG_DFL

Comportamiento por defecto.

SIG_IGN

Ignorar la señal.

handler

Código personalizado (función manejadora).

Buenas prácticas

- Hacer el mínimo trabajo posible.
- Evitar I/O y funciones no seguras.
- Usar `sig_atomic_t` para banderas compartidas.

¿Cómo termina un proceso?

Terminación normal

- El programa llama a `exit(código)`.
- `main()` retorna.

Terminación anormal

- Señal no manejada: `SIGSEGV`, `SIGBUS`, `SIGFPE`.
- `abort()` envía `SIGABRT` (produce un core).
- `SIGKILL` termina sin poder ser bloqueada.

0 Éxito

≠ 0 Error

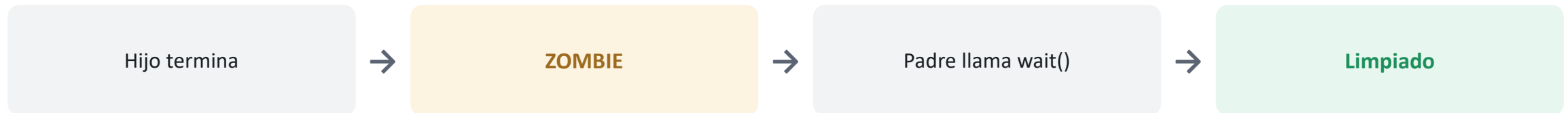
El shell expone este valor en la variable `$?` después de cada comando.

Esperando la terminación: `wait()`

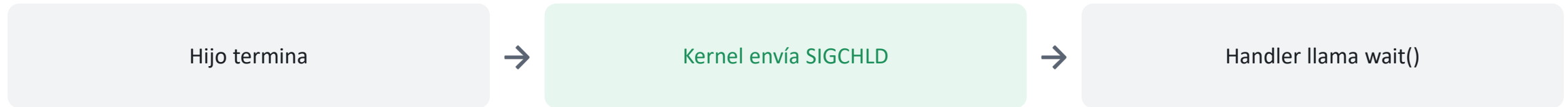
- `wait()` bloquea al padre hasta que un hijo termine.
- `WIFEXITED` / `WEXITSTATUS` revelan cómo terminó el hijo.
- `waitpid()` espera a un hijo específico.
- `wait3()` / `wait4()` añaden estadísticas de uso de CPU.

Proceso zombie

- Un hijo que terminó pero cuyo padre aún no llamó a `wait()`.
- Sigue ocupando una entrada en la tabla de procesos.
- Si el padre nunca lo limpia, `init` (PID 1) lo hereda y lo limpia.



Limpieza asíncrona con SIGCHLD



- Evita bloquear al padre esperando con wait().
- wait3() / wait4() con WNOHANG no bloquean si no hay hijos terminados.
- Patrón común: manejar SIGCHLD y limpiar dentro del handler.

```
void clean_up(int signum) {  
    int status;  
    wait(&status);  
    child_exit_status = status;  
}  
  
sigchld_action.sa_handler = &clean_up;  
sigaction(SIGCHLD, &sigchld_action, NULL);
```

Resumen



Introducción

PID y PPID identifican procesos; init es la raíz del árbol.



Creación

fork() duplica, exec() reemplaza; juntos forman el patrón spawn.



Señales

Comunicación asíncrona; sigaction() define cómo se manejan.



Terminación

exit() y wait() evitan que queden procesos zombie.

Referencia: Advanced Linux Programming, Capítulo 3 — Mitchell, Oldham & Samuel

FIN DEL CAPÍTULO 3 · PROCESOS

¿Preguntas?

Gracias — System Programming